

# Design Note: LLD Coach

This document explains the architecture, domain models, design patterns, and design decisions behind **LLD Coach** in simple terms.

## 1. Architecture Overview

LLD Coach is built as a simple, modular monolithic application with a clean layered architecture:



### Layer Responsibilities:

- **Frontend (React + Vite + Tailwind CSS):** Provides the user interface for browsing problems, practicing solutions in a structured editor, and viewing explainable feedback.
- **Controllers (Express):** Handle incoming HTTP requests, validate input data, and return JSON responses.
- **Services:** Contain the core application workflows (managing problems, handling practice attempts, and triggering evaluations).
- **Repositories:** Encapsulate all database operations for problems and attempts.
- **Evaluation Engine:** Evaluates candidate submissions using deterministic checks and AI analysis.

## 2. Main Domain Models

The core domain flow is structured as:

**Problem → Attempt → Submission → EvaluationResult**

### 1. Problem

Represents an LLD practice problem (e.g., *Parking Lot*, *Vending Machine*).

- Stores the title, difficulty, functional requirements, expected core entities, starter template, and evaluation rubric.

### 2. Attempt

Represents a user's practice session for a problem.

- Tracks the problem being solved, user draft progress, and the list of submissions.

### 3. Submission

Represents a specific snapshot of a solution submitted by the user.

- Contains the 4 design sections:
  1. *Design Explanation*
  2. *Classes & Responsibilities*
  3. *Code / Pseudocode*
  4. *Trade-offs & Assumptions*
- Holds the evaluation status (PENDING, EVALUATING, COMPLETED, FAILED).

### 4. EvaluationResult

Represents the feedback generated for a submission.

- Contains the overall score (0–100), verdict, category scores (SOLID, Class Design, Extensibility, Edge Cases), strengths, issues, suggestions, and a recommended next improvement.

## 3. Design Patterns Used

### 1. Strategy Pattern (IEvaluator)

- **Why it is used:** Allows different evaluation approaches to be used interchangeably.
- **Interface:**

```
export interface IEvaluator { evaluate(problem: Problem, submission: Submission): Promise<EvaluationResult>; }
```

- **Implementations:**
  - **DeterministicEvaluator:** Checks for mandatory sections, word count, core problem entities, and design pattern keywords.
  - **AIEvaluator:** Evaluates SOLID principles, abstraction quality, and missing edge cases.
  - **FallbackEvaluator:** Generates rule-based feedback if the AI service fails or times out.

- **EvaluationPipeline:** Coordinates the evaluation by running deterministic checks first, then AI evaluation with fallback.

## 2. Adapter Pattern (ILLMProvider)

- **Why it is used:** Decouples the AI evaluator from any single AI vendor.
- **Interface:**

```
export interface ILLMProvider { generateCompletion(prompt: string, options?: LLMCompletionOptions): Promise<string>; }
```

- **Implementations:**
  - **GeminiProvider:** Connects to Google Gemini API.
  - **OpenAIPProvider:** Connects to OpenAI API.
  - **MockProvider:** Provides simulated AI feedback for offline development and testing.

## 3. Repository Pattern

- **Why it is used:** Separates business logic from database queries.
- **ProblemRepository** and **AttemptRepository** handle database reading and writing, keeping the service layer clean and easy to test.

## 4. Important Design Decisions

---

### 1. Modular Monolith instead of Microservices:

The application is kept as a single, well-organized project. This avoids unnecessary distributed system complexity and keeps the project easy to build, test, and run locally.

### 2. Structured 4-Section Editor instead of One Large Text Box:

Dividing practice into *Explanation*, *Classes*, *Code*, and *Trade-offs* helps learners organize their object-oriented thoughts step by step and allows the evaluators to analyze each section accurately.

### 3. Synchronous Evaluation with Timeout & Fallback:

Evaluation is handled synchronously for this small application. A timeout and fallback mechanism are used so that if AI evaluation is unavailable or fails, the system can still provide basic feedback.

## 5. Extensibility

---

The codebase is designed to be easily extended:

- **Adding a New Evaluator:** Implement the **IEvaluator** interface and add the new evaluator to **EvaluationPipeline**.
- **Adding a New AI Provider:** Implement the **ILLMProvider** interface and register it in the provider resolver.